BÀI 6: BẢO VỆ MÁY CHỦ VÀ TỰ ĐỘNG HÓA CÁC YÊU CẦU WEB

Bài thực hành này giúp sinh viên tiếp cận các kỹ thuật bảo vệ máy chủ web trước các yêu cầu bất thường và tấn công tự động, đồng thời hiểu cách tự động hóa gửi yêu cầu HTTP để kiểm thử hoặc khai thác lỗ hổng. Sinh viên sẽ làm việc với công cụ mô phỏng hành vi của client tự động và cấu hình bảo mật cơ bản để giám sát, giới hạn và ngăn chặn các truy cập có hại đến máy chủ.

6.1 CHUẨN BỊ

Sử dụng hai máy Linux để thống nhất câu lệnh:

Máy	**Hệ điều hành**	**Vai trò**
Máy A	Ubuntu hoặc Kali Linux	Máy chủ, cấu hình tường lửa
Máy B	Kali Linux hoặc Ubuntu	Máy khách, gửi yêu cầu kiểm thử

Các cổng triển khai:

Cổng	**Chức năng**
TCP 9999	Kết nối thử nghiệm bằng Netcat
TCP 8080	Máy chủ HTTP nội bộ

Thực hiện cài phần mềm trước khi cô lập mạng. Có thể tạm thời dùng card NAT để tải gói, sau đó ngắt card NAT khi thực hành.

Tại máy A:

```
sudo apt update
sudo apt install -y iptables iptstate iproute2 netcat-openbsd python3 curl nano
```

Tại máy B:

```
sudo apt update
sudo apt install -y iproute2 netcat-openbsd python3 curl nano
```

Mở Terminal kiểm tra cài đặt:

```
python3 --version
iptables --version
command -v iptstate
command -v nc
```

6.2 BẢO VỆ MÁY CHỦ THÔNG QUA IPTABLES VÀ IPTSTATE

6.2.1 Mục đích

Để bảo vệ máy chủ bằng tường lửa **iptables** và giám sát các kết nối bằng **iptstate**. Tường lửa là một biện pháp phòng thủ rất mạnh mẽ và mọi máy chủ nên sử dụng tường lửa.

6.2.2 Cần nắm các yêu cầu sau

Các chuỗi cơ bản:

Chuỗi	**Vai trò**
INPUT	Xử lý gói tin có đích đến là máy hiện tại
OUTPUT	Xử lý gói tin do máy hiện tại tạo ra
FORWARD	Xử lý gói tin đi xuyên qua máy khi máy làm nhiệm vụ chuyển tiếp

Các trạng thái và hành động kết nối:

Trạng thái	**Ý nghĩa khái quát**
NEW	Thuộc kết nối mới
ESTABLISHED	Thuộc kết nối đã có lưu lượng trao đổi hai chiều
RELATED	Liên quan đến một kết nối đã được theo dõi
INVALID	Không xác định được trạng thái hợp lệ
Hành động	
ACCEPT	Cho phép gói tin
DROP	Loại bỏ gói tin, không chủ động phản hồi lỗi
REJECT	Từ chối và gửi phản hồi lỗi phù hợp

6.2.3 Hướng dẫn

Kiểm tra mạng:

Đứng tại hai máy kiểm tra địa chỉ IP:

```
ip -br address
ip route
```

Ping kết nối hai máy:

```
ping -c 4 <IP_kali>
```

Ping thành công chỉ chứng minh việc trao đổi ICMP hoạt động. Nó không chứng minh máy chủ đang chạy dịch vụ web hoặc cổng HTTP đã được cho phép.

Ghi nhận cấu hình ban đầu:

Trên máy A, mở terminal nhập lệnh sau:

```
sudo iptables -S
sudo iptables -L -n -v --line-numbers
sudo ss -lntup
```

Sao lưu cấu hình IPv4:

```
sudo iptables-save > lab06<yourname>-before.rules
```

Không ghi đè tệp sao lưu này sau khi đã bắt đầu thay đổi cấu hình.

Tạo dịch vụ kiểm thử:

Tại máy A:

```
nc -l -v -p 9999
```

Giữ cửa sổ này. Mở trong cửa sổ khác:

```
sudo ss -lntp 'sport = :9999'
```

Kết quả có socket ở trạng thái `LISTEN` trên cổng 9999.

Từ máy B:

```
nc -v <IP_mayA> 9999
```

Nhập nội dung sau:

```
<YOURNAME> - Kiem tra ket noi ban dau
```

Kết quả máy A nhận được dòng vừa nhập. Nếu chưa kết nối được, phải xử lý trước khi thêm quy tắc mới. Kiểm tra IP, dịch vụ lắng nghe và tường lửa sẵn có.





Tạo chuỗi tường lửa riêng:

Trên máy A, đặt biến bằng tên card Host-only thực tế:

```
LAB_IF=<ten mang>
```

Kiểm tra lại thông tin:

```
ip -br address show dev "$LAB_IF"
```

Tạo chuỗi:

```
sudo iptables -N LAB06_IN
```



Nếu báo chuỗi đã tồn tại, dừng và kiểm tra lần thực hành trước. Không tiếp tục thêm quy tắc trùng lặp.

Thêm các quy tắc sau:

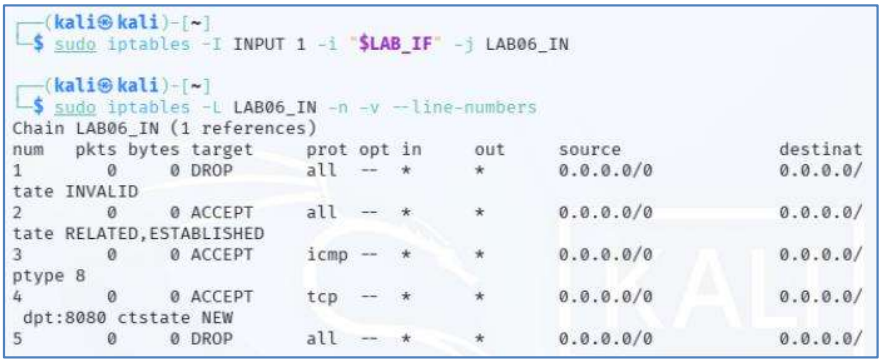
```
sudo iptables -A LAB06_IN -m conntrack --ctstate INVALID -j DROP
sudo iptables -A LAB06_IN -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
sudo iptables -A LAB06_IN -p icmp --icmp-type echo-request -j ACCEPT
sudo iptables -A LAB06_IN -p tcp --dport 8080 -m conntrack --ctstate NEW -j ACCEPT
sudo iptables -A LAB06_IN -j DROP
```

Áp dụng chuỗi vào lưu lượng đi vào từ card Host-only:

```
sudo iptables -I INPUT 1 -i "$LAB_IF" -j LAB06_IN
```

Kiểm tra:

```
sudo iptables -L LAB06_IN -n -v --line-numbers
```



Các bước cài đặt trên nhằm:

- Loại bỏ gói có trạng thái không hợp lệ.

- Cho phép lưu lượng thuộc kết nối đã thiết lập hoặc liên quan.
- Cho phép ping.
- Cho phép kết nối TCP mới đến cổng 8080.
- Chặn lưu lượng còn lại đi qua chuỗi.

Cấu hình này chỉ tác động đến lưu lượng đầu vào trên card Host-only được chọn, không phải chính sách bảo vệ toàn bộ các card mạng của máy.

Kiểm tra cổng bị chặn:

Từ máy B:

```
ping -c 4 <IP_mayA>
```

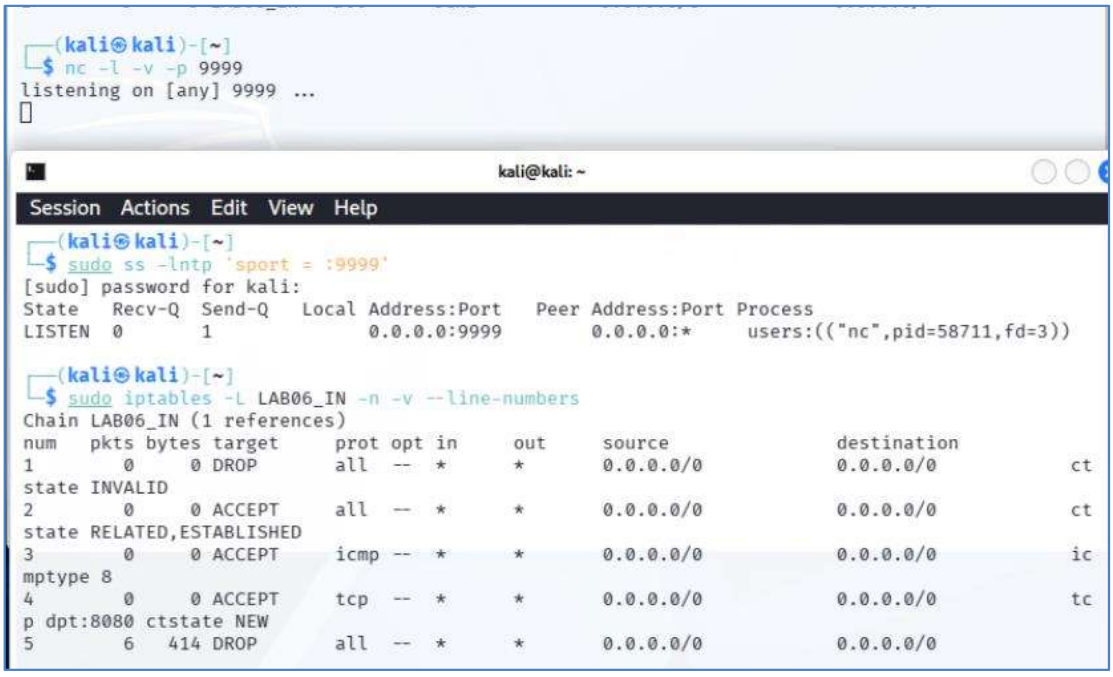
Tiếp theo nhập lệnh sau, kiểm tra kết nối đến cổng 9999:

```
Nc -n -v -z -w 3 <IP_mayA> 9999
```



Kết nối mới đến cổng 9999 không thành công, thường hết thời gian chờ. Tại terminal, trên máy A:

```
sudo iptables -L LAB06_IN -n -v --line-numbers
```



Quan sát bộ đếm gói của quy tắc DROP cuối chuỗi.

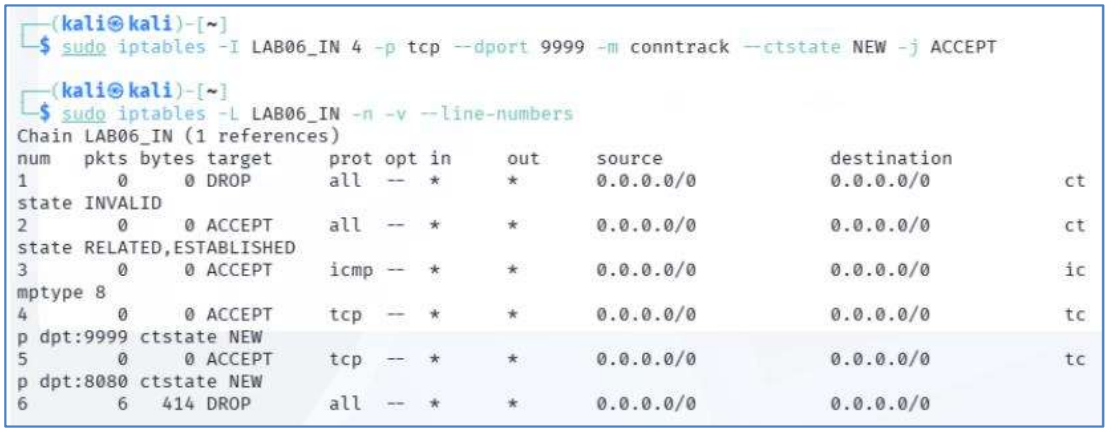
Yêu cầu: giải thích vì sao ping thành công nhưng TCP 9999 thất bại.

Cho phép cổng 9999:

Đứng tại máy A:

```
sudo iptables -I LAB06_IN 4 -p tcp --dport 9999 -m conntrack --ctstate NEW
-j ACCEPT
```

Lệnh chèn quy tắc trước quy tắc DROP cuối chuỗi.

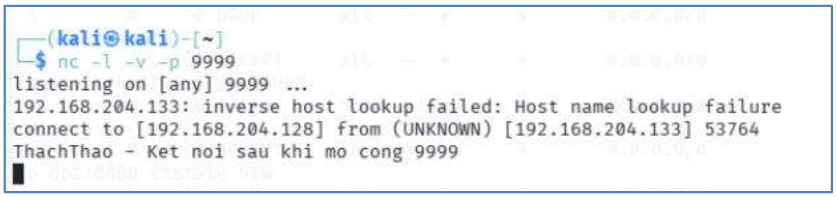


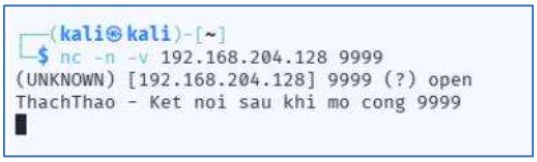
Phải thấy quy tắc `ACCEPT tcp dpt:9999` đứng trước DROP cuối chuỗi.

Tại máy B và nhập:

```
nc -n -v 192.168.56.10 9999
LAB06 - Ket noi sau khi mo cong
```

Kết quả lúc này, máy A nhận dữ liệu. Giữ kết nối mở để quan sát ở bước tiếp theo.





Quan sát bằng iptstate:

Trên máy chủ Linux, trong cửa sổ Terminal, hãy thực hiện lệnh sau:

```
sudo iptstate
```

Tìm kết nối có:

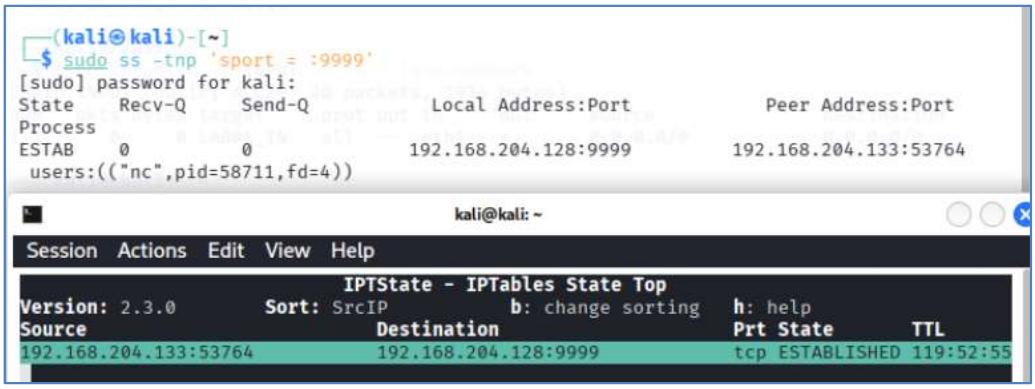
- Địa chỉ máy khách: `192.168.56.20`.
- Cổng dịch vụ: `9999`.
- Giao thức: `TCP`.
- Trạng thái: `ESTABLISHED`.



Đối chiếu bằng:

```
sudo ss -tnp 'sport = :9999'
```

Lưu ý: iptstate hiển thị bảng theo dõi kết nối; `ss` hiển thị socket. Hai công cụ có góc quan sát khác nhau, không nhất thiết hiển thị danh sách giống hệt nhau.



Phân biệt kết nối hiện có và kết nối mới:

Trên máy A, xóa quy tắc cho phép cổng 9999:

```
sudo iptables -D LAB06_IN -p tcp --dport 9999 -m conntrack --ctstate NEW -j
ACCEPT
```

Từ máy B, tạo kết nối mới:

```
nc -4 -vz -w 3 <IP_mayA> 9999
```

Kết quả hiển thị, kết nối mới bị chặn.

Nếu kết nối cũ chưa đóng, nó có thể vẫn hoạt động do quy tắc ESTABLISHED,RELATED. Vì vậy, khi kiểm tra thay đổi quyền truy cập, phải phân biệt kết nối mới với kết nối đã tồn tại.

Lưu ý: Giữ nguyên quyền truy cập cổng 8080 để thực hiện phần 6.2.

6.3 TỰ ĐỘNG HÓA CÁC YÊU CẦU WEB BẰNG PYTHON

6.3.1 Mục đích

Thực hành quan sát yêu cầu HTTP được tạo từ trình duyệt, xác định thành phần của yêu cầu và sử dụng Python 3 để gửi lại yêu cầu đến website thực hành.

Sau khi hoàn thành, sinh viên có khả năng:

- Xác định URL, phương thức, header và dữ liệu biểu mẫu.
- Quan sát yêu cầu, phản hồi bằng công cụ phát triển của trình duyệt.
- Tạo yêu cầu POST bằng Python 3.
- Đọc mã trạng thái, nội dung và đo thời gian phản hồi.
- Giải thích hạn chế của việc suy luận từ thời gian phản hồi.

6.3.2 Hướng dẫn

Cấu hình mạng máy ảo:

Giữ card Host-only nếu còn cần dùng. Bổ sung hoặc bật card mạng NAT, sau đó kiểm tra địa chỉ và tuyến mạng:

```
ip -br address
ip route
```

Kiểm tra phân giải tên miền:

```
getent ahostsv4 attackdirect.samsclass.info
```

Nếu không có kết quả, kiểm tra kết nối NAT và cấu hình DNS.

Kiểm tra truy cập trang:

```
curl -I --connect-timeout 5 --max-time 15
http://attackdirect.samsclass.info/blind.htm
```

Lệnh trên gửi HEAD để kiểm tra sơ bộ. Nếu máy chủ không hỗ trợ HEAD, hãy kiểm tra bằng Firefox.

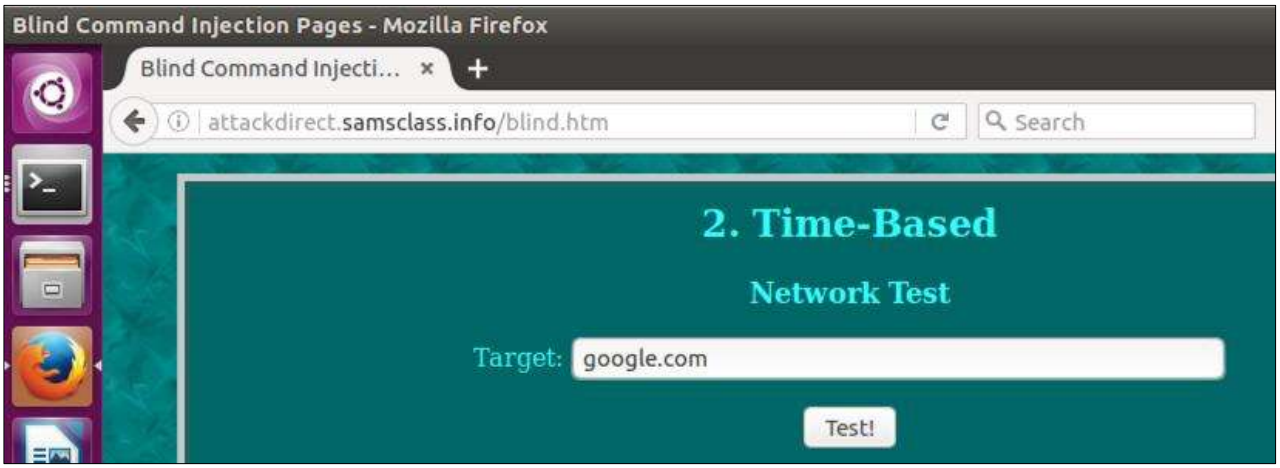
Kiểm tra OUTPUT:

```
sudo iptables -S OUTPUT
```

Nếu còn quy tắc chặn TCP mới đi ra, cần khôi phục cấu hình theo hướng dẫn giảng viên trước khi tiếp tục. Không xóa toàn bộ tường lửa.

Truy cập website và xác định bài thực hành:

Từ máy tính Linux, khởi chạy Firefox. Trong Firefox, hãy truy cập: http://attackdirect.samsclass.info/blind.htm. Cuộn xuống mục "2. Time-Based", như hình bên dưới.



Quan sát yêu cầu HTTP:

Trong Firefox, thực hiện lần lượt các bước sau:

1. Nhấn **F12**.
2. Chọn thẻ **Network**.
3. Bật **Persist Logs** nếu cần giữ yêu cầu qua lần chuyển trang.
4. Xóa danh sách yêu cầu cũ để dễ quan sát.

Tiếp theo, tìm ví dụ thứ ba trong mục **Time-Based**, sao chép trực tiếp ví dụ hiện được website cung cấp vào ô **Target**. Nhấn **Test!** một lần và chờ website trả kết quả.

Trong Network, tìm yêu cầu tương ứng phát sinh từ nút **Test!**

```
Phương thức: POST
Đường dẫn: /blind2.php
```

Nếu **Network** hiển thị **Form Data**, sao chép giá trị đã giải mã của trường `target`. Không lấy toàn bộ chuỗi dạng:

```
target=...%xx...
```

rồi đưa nguyên chuỗi đó vào giá trị `target` trong Python. Làm như vậy có thể mã hóa hai lần và khiến yêu cầu khác với yêu cầu ban đầu.

Chuẩn bị Python 3:

Kiểm tra Py:

```
python3 --version
```

Nếu chưa có:

```
sudo apt update
sudo apt install -y python3 nano
```

Tạo thư mục thực hành:

```
mkdir -p ~/lab06
cd ~/lab06
```

Tạo chương trình gửi lại yêu cầu POST:

Mở và tạo tệp sau:

```
nano replay_request.py
```

Nhập vào chương trình:

```
#!/usr/bin/env python3
import time
from urllib.parse import urlencode
from urllib.request import Request, urlopen
from urllib.error import HTTPError, URLError

# Đối chiếu với Request URL thực tế trong Firefox trước khi chạy.
URL = "http://attackdirect.samsclass.info/blind2.php"

```

```
# Nhập giá trị đã giải mã của trường target.
# Chỉ dùng ví dụ của website được giảng viên cho phép.
target = input("Nhap gia tri truong target: ")

form_data = {
    "target": target
}

body = urlencode(form_data).encode("utf-8")

request = Request(
    URL,
    data=body,
    method="POST",
    headers={
        "Content-Type": "application/x-www-form-urlencoded",
        "User-Agent": "LAB06-<YOURNAME>-Client/1.0",
        "Accept-Encoding": "identity"
    }
)

start = time.perf_counter()

try:
    with urlopen(request, timeout=30) as response:
        raw = response.read()
        elapsed = time.perf_counter() - start
        charset = response.headers.get_content_charset() or "utf-8"

        print("\n--- RESPONSE ---")
        print("Final URL:", response.geturl())
        print("Status:", response.status)
        print("Content-Type:", response.headers.get("Content-Type"))
        print("Content-Encoding:",response.headers.get("Content-Encoding"))
        print("Elapsed:", f"{elapsed:.3f}", "seconds")
        print("\n--- BODY ---")
```

```
        print(raw.decode(charset, errors="replace"))

except HTTPError as error:
    print("HTTP error:", error.code)
    print(error.read().decode("utf-8", errors="replace"))
    error.close()

except (URLError, TimeoutError) as error:
    print("Connection error:", error)
```

Lưu bằng **Ctrl+O**, **Enter**; thoát bằng **Ctrl+X**.

Chạy và đối chiếu kết quả:

```
python3 replay_request.py
```

Chương trình chỉ gửi một yêu cầu, không tạo vòng lặp.